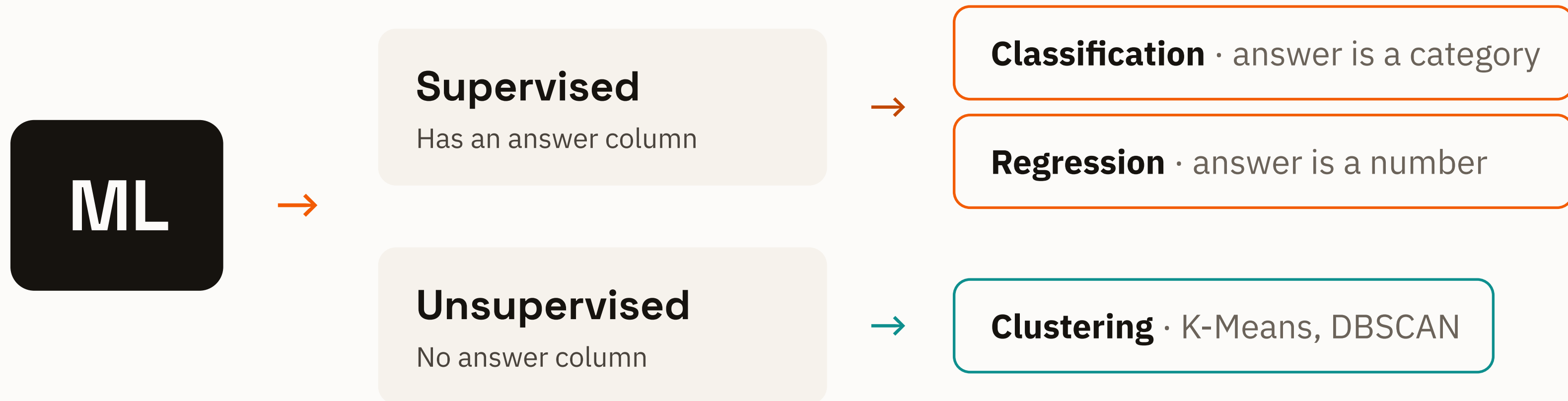


# Roadmap to Learn NLP for Machine Learning

How text becomes numbers a model can learn from, and the order of topics from cleaning text to BERT.

RECAP

# Where We Are in Machine Learning



# Independent Features In, Dependent Feature Out

| Input · independent features |    |    |     | Output · dependent |
|------------------------------|----|----|-----|--------------------|
| f1                           | f2 | f3 | f4  | y                  |
| 2.1                          | 45 | 0  | 7.3 | 1                  |
| 3.4                          | 31 | 1  | 5.0 | 0                  |
| 1.8                          | 58 | 1  | 6.6 | 1                  |

NOTICE

Every cell is a **number** . Models can only do maths on numbers.

WHEN THE INPUT IS TEXT

# Spam Classification: Spam or Not Spam?

| EMAIL SUBJECT    | EMAIL BODY                                   | SPAM / HAM |
|------------------|----------------------------------------------|------------|
| Billionaire      | "You won a lottery of a billion dollars"     | 1 • spam   |
| Team meeting     | "Can we move tomorrow's meeting to 3 pm?"    | 0 • ham    |
| Claim your prize | "Click this link now to get your free phone" | 1 • spam   |

The output is still a number. The **inputs are sentences**, and a model cannot do maths on words.

## THE CORE JOB

# Input Text → Vectors → Model

"You won a lottery of a billion dollars"



[0, 1, 1, 0, 1, 1, 0]



ML model  
1 · spam

A **vector** is a list of numbers. Once text is a vector, every algorithm we already know can use it.

# Products Built on NLP

01

## Alexa

"Alexa, play some music." It turns your words into a command it can act on.

02

## Google Home

"What is the weather today?" It understands the question and replies in a sentence.

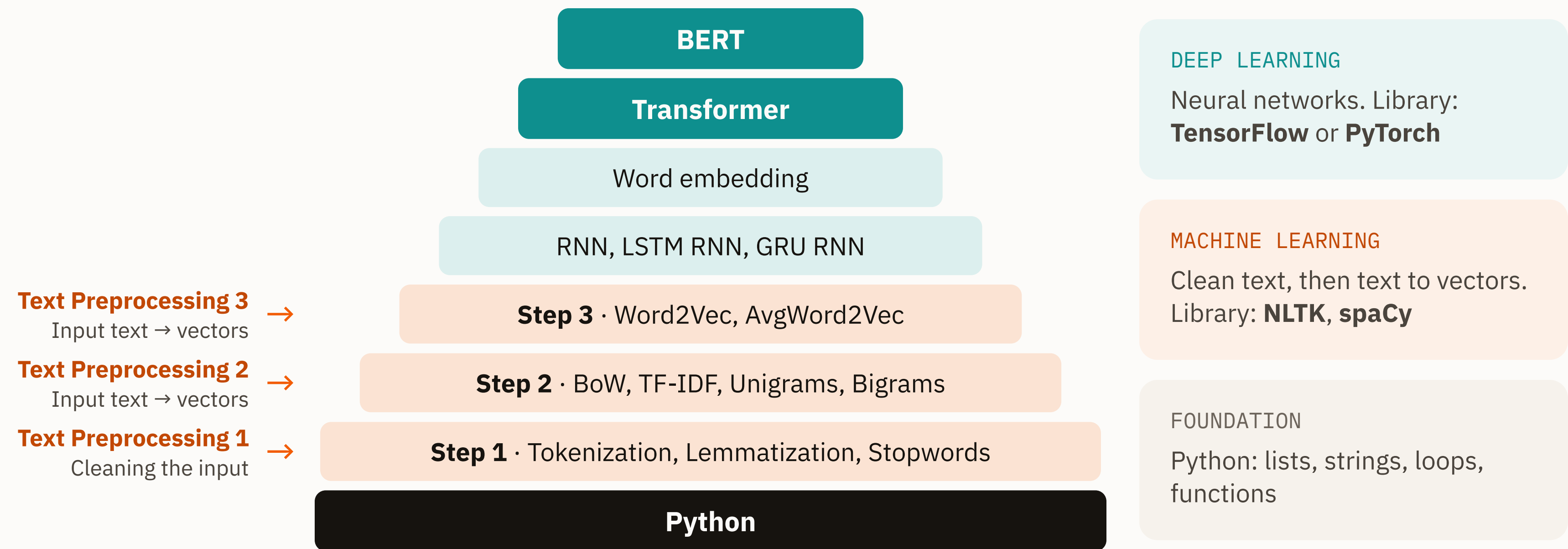
THE ROADMAP

# Roadmap of NLP

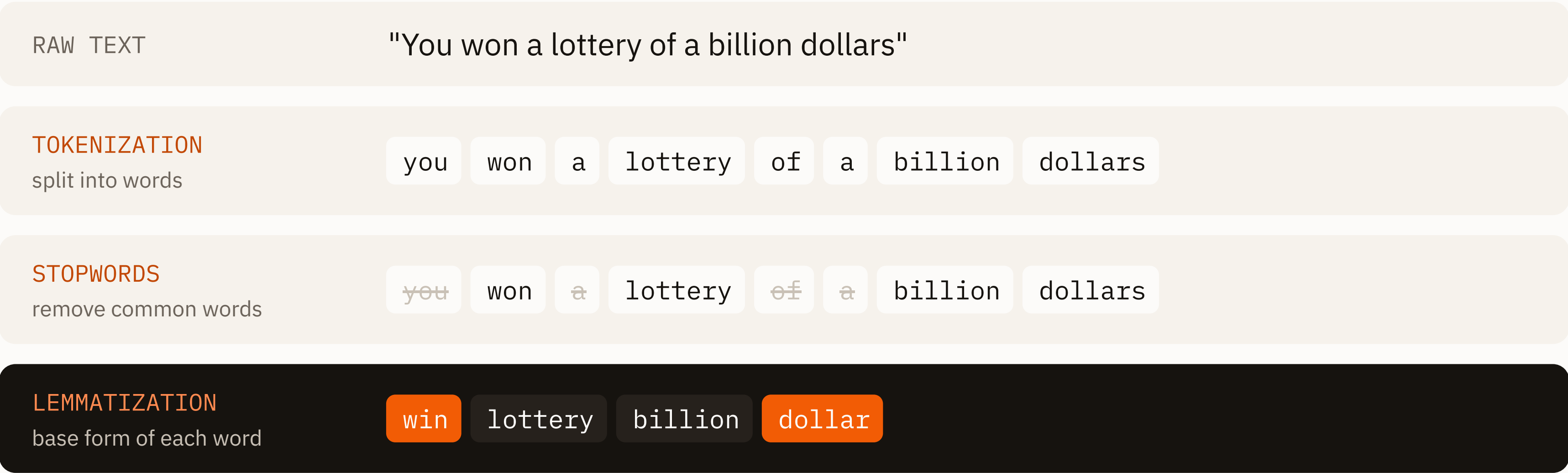
Start at the bottom with Python and climb one step at a time. Each step is a better way to turn text into vectors.

READ FROM THE BOTTOM UP

# The NLP Roadmap



# Cleaning the Input Text



# Text to Vectors by Counting Words

BAG OF WORDS

|         | win | lottery | billion | meeting | move |
|---------|-----|---------|---------|---------|------|
| Email 1 | 1   | 1       | 1       | 0       | 0    |
| Email 2 | 0   | 0       | 0       | 1       | 1    |

Each email becomes a row of word counts.

TF-IDF

Counts, but rare words get a higher weight than common ones.

UNIGRAMS AND BIGRAMS

Unigram: billion. Bigram: billion dollar , a pair of neighbouring words.

# Word2Vec: Vectors That Carry Meaning

## WORD2VEC

|         |                    |
|---------|--------------------|
| lottery | [0.81, 0.12, 0.64] |
|---------|--------------------|

|       |                    |
|-------|--------------------|
| prize | [0.78, 0.15, 0.60] |
|-------|--------------------|

|         |                    |
|---------|--------------------|
| meeting | [0.05, 0.90, 0.21] |
|---------|--------------------|

Similar meaning, similar numbers.

## AVGWORD2VEC

Average the vectors of every word in the email to get **one vector per email**.

```
(win + lottery + billion + dollar) / 4
```

# The Neural Network Half of the Roadmap

## 04 · RNN, LSTM RNN, GRU RNN

Read a sentence word by word and remember earlier words. Word order now matters.

## 05 · Word embedding

Word vectors learned inside the network, tuned for the task at hand.

## 06 · Transformer

Reads all words at once and learns which words to pay attention to.

## 07 · BERT

A large transformer already trained on huge amounts of text. We fine-tune it for our own problem.

## SUMMARY

# What to Remember

01 Models need numbers. NLP turns **text into vectors**.

02 ML half: clean, then BoW / TF-IDF, then Word2Vec. Uses **NLTK, spaCy**.

03 DL half: RNN family, embeddings, transformer, BERT. Uses **TensorFlow / PyTorch**.

→ Next: Step 1 in code, tokenization, stopwords and lemmatization.